

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview

This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
 - Complete Size Counting Standard
-

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
 - provide criteria for judging the quality of the code that you produce
 - facilitate size counting by ensuring your programs are written so they can be readily counted
 - for LOC counting, require that there be a separate physical line for each logical line of code
-

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Proyecto: Analizador de Código Java (LOC y Métodos) — Versión PSP 0.1

Fecha: 2025-11-05

Objetivo

Definir un estándar de codificación específico para el programa actual (App, Logic2, Input, Data, LineCounter, MethodCounter, Output y el programa de ejemplo DesvEst) que asegure: legibilidad, mantenibilidad y compatibilidad con el estándar de conteo de tamaño (R2).

1. Paquetes, estructura y codificación

- Paquete único para el analizador: package analyzer; La estructura de carpetas debe ser src/main/java/analyzer/ con un archivo por clase.
- Codificación de texto: UTF-8.
- Un archivo .java por clase pública, el nombre del archivo debe coincidir con el nombre de la clase.
- Importaciones agrupadas y sin usar deben eliminarse.

2. Encabezado de programa por módulo

Todos los archivos deben iniciar con el siguiente bloque, personalizado por módulo. Ejemplos concretos para este proyecto:

```
/******  
* Programa: App.java  
* Paquete: analyzer  
* Autor: <Jared Fernández González>  
* Fecha: 2025-11-05  
* Descripción: Ejecuta el flujo: lee archivo objetivo, invoca Logic2 y produce el  
reporte.  
*****/
```

```

/*****
* Programa:    Logic2.java
* Paquete:     analyzer
* Autor:       <Jared Fernández González>
* Fecha:       2025-11-05
* Descripción:  Coordina Input, Data, LineCounter y MethodCounter. Controla
validaciones y salida.
*****/

```

```

/*****
* Programa:    Input.java
* Paquete:     analyzer
* Autor:       <Jared Fernández Gozález>
* Fecha:       2025-11-05
* Descripción:  Lee el archivo objetivo desde disco y expone su contenido
como String.
*****/

```

```

/*****
* Programa:    Data.java
* Paquete:     analyzer
* Autor:       <Jared Fernández González>
* Fecha:       2025-11-05
* Descripción:  Normaliza saltos de línea y separa en arreglo de líneas.
*****/

```

```

/*****
* Programa:    LineCounter.java
* Paquete:     analyzer
* Autor:       <Jared Fernández González>
* Fecha:       2025-11-05
* Descripción:  Cuenta LOC lógicas excluyendo comentarios y líneas vacías
(ver R2).
*****/

```

```

/*****
* Programa:    MethodCounter.java
* Paquete:     analyzer
* Autor:       <Jared Fernández González>
* Fecha:       2025-11-05
* Descripción:  Detecta y cuenta firmas de métodos/constructores.
*****/

```

```

/*****
* Programa:    Output.java
* Paquete:     analyzer
* Autor:       <Jared Fernández González >
* Fecha:       2025-11-05
* Descripción:  Escribe los resultados a un archivo de salida (reporte).
*****/

```

```

/*****
* Programa:    DesvEst.java
* Paquete:     analyzer
* Autor:       <Jared Fernández González>
* Fecha:       2025-11-05
* Descripción: Programa de ejemplo para ser analizado por el sistema
(cálculo de desviación estándar).
*****/

```

3. Convenciones de nombres

- Clases en PascalCase. Métodos y variables en lowerCamelCase. Constantes en UPPER_SNAKE_CASE.
- Nombres significativos, evitar abreviaturas crípticas y variables de una letra.
- Mapeo concreto para este proyecto:

Actual	Debe ser
lineCounter	LineCounter
methodCounter	MethodCounter
output	Output
Logic2	Logic (opcional) o AnalyzerLogic

4. Estilo y formato

- Una instrucción lógica por línea física para facilitar el conteo de LOC.
- Llaves: estilo Allman permitido, pero preferir K&R consistente en todo el proyecto.
- Longitud máxima de línea: 120 caracteres.
- Indentación: 4 espacios, sin tabs.

- Espaciado: un espacio después de comas y alrededor de operadores binarios.
- Líneas con sólo '{' o '}': no cuentan como LOC (ver R2).

Ejemplo — Antes:

```
int a=0; int b=1; if(a<b){b=a;} // mala práctica
```

Ejemplo — Después:

```
int a = 0;
int b = 1;
if (a < b) {
    b = a;
}
```

5. Comentarios y documentación

- Comentarios de bloque para describir el propósito de clases y métodos públicos.
- Comentarios en línea sólo para aclaraciones no triviales; evitar comentar lo obvio.
- No se cuentan para LOC.
- En LineCounter y MethodCounter documentar explícitamente los casos límite cubiertos:
 - Comentarios de línea y bloque, y comentarios dentro de strings.
 - Firmas con anotaciones, genéricos y throws.
 - Llave en línea siguiente.
- Javadoc mínimo en clases públicas y métodos públicos.

6. Reglas específicas por módulo

App

- Debe aceptar parámetros de entrada: ruta del archivo a analizar y ruta del reporte de salida.
- Validar existencia/legibilidad del archivo y mostrar mensaje de uso con --help.
- Delegar toda la lógica a Logic2; no mezclar I/O con reglas de conteo.

Logic2

- Orquestar Input → Data → LineCounter/MethodCounter → Output.
- Manejar excepciones y producir mensajes claros (ruta absoluta, causa).
- No mantener campos no usados; evitar estado mutable innecesario.

Input

- Usar NIO: Files.readString(Path, UTF_8) o try-with-resources con BufferedReader.
- Declarar explícitamente la codificación UTF-8.

Data

- Normalizar finales de línea a \n.
- Regresar String[] sin nulos; entradas vacías deben producir arreglo vacío.

LineCounter

- Eliminar comentarios de bloque preservando saltos de línea.
- Remover // sólo si está fuera de literales de cadena.
- No contar líneas vacías ni líneas con sólo llaves.
- Proveer método count(String code): int y/o count(String[] lines): int.

MethodCounter

- Detectar firmas con anotaciones y genéricos.
- Considerar constructores (sin tipo de retorno).

- Soportar '{' en la siguiente línea.
- Excluir if/for/while/switch/catch/else.
- Exponer countMethods(String[] lines): int.

Output

- Usar NIO para escribir el reporte; crear directorios si no existen.
- Formato del reporte: tabla con Archivo, LOC (sin comentarios) y Métodos.

DesvEst (programa analizado)

- Validar $n > 1$ y `dataList.length >= n`.
- Manejar `NumberFormatException`.
- Especificar si la desviación es muestral ($n-1$) o poblacional (n).

7. Manejo de errores y mensajes

- Evitar `printStackTrace` en producción; preferir mensajes claros y, si aplica, código de salida distinto de cero.
- Mensajes deben incluir ruta del archivo y causa. Ej.: "No se pudo leer /ruta/archivo.java: Permiso denegado".

8. Pruebas unitarias y de integración

- Casos unitarios mínimos para `LineCounter` y `MethodCounter`:
 - Comentarios de línea y bloque, y comentarios dentro de strings.
 - Firmas con `@Override`, genéricos (`<T>`), `throws`, y llave en nueva línea.
 - Interfaces con métodos terminados en `';`.
- Prueba de integración: ejecutar `Logic2` sobre un conjunto de 5–10 archivos mixtos y verificar resultados esperados.
- Todo cambio en las reglas debe venir acompañado de nuevas pruebas.

9. Compatibilidad con Size Counting Standard (R2)

- Este R1 está alineado para facilitar el conteo de LOC lógicas definido en R2.
- Se prohíbe escribir múltiples sentencias en una sola línea.
- Las anotaciones y líneas con llaves solas no suman LOC.

10. Checklist de R1 para este programa

- Clases renombradas a PascalCase (LineCounter, MethodCounter, Output).
- package analyzer; presente y estructura de carpetas aplicada.
- App recibe rutas por argumentos y valida entrada.
- LineCounter ignora comentarios y líneas vacías; strings respetados.
- MethodCounter contempla anotaciones, genéricos y constructores.
- Una sentencia por línea; llaves solas no cuentan LOC.
- Reporte con Archivo, LOC, Métodos escrito con NIO.
- Pruebas para casos límite agregadas.